

An aerial photograph of a city skyline. In the foreground, a dark, rectangular skyscraper with a flat roof is prominent. The roof has several circular vents and a small structure. To the left, a lighter-colored building with a grid-like window pattern is visible. To the right, another dark building with a similar grid pattern is seen. The background shows more buildings and a hazy sky.

# Aula 6

## Transformers: Mecanismo de Atenção

Tópicos Avançados em Inteligência Artificial · UFABC

# Roteiro da aula

**Parte 1 — Motivação** Polisssemia, tarefa ATIS e requisitos de uma solução

**Parte 2 — Self-Attention** Representação contextual, produto interno, softmax

**Parte 3 — Multi-Head Attention** Múltiplas cabeças paralelas, concatenação e projeção

**Parte 4 — Positional Encoding** Problema da permutabilidade, embeddings posicionais

**Parte 5 — Encoder Completo** Q/K/V, conexões residuais, LayerNorm, bloco completo

**Parte 6 — Aplicações** Slot filling, tipos de tarefas, código PyTorch

# Parte 1 — Motivação

# De RNNs para Transformers

Na aula anterior vimos RNNs e LSTMs como modelos para sequências. Por que não são suficientes para NLP moderno?

## Processamento sequencial

RNNs calculam  $h_t$  a partir de  $h_{t-1}$  — impossível paralelizar. N tokens = N passos em série, mesmo em GPUs.

❌ Treinamento lento em sequências longas

## Vanishing gradient

Gradientes se dissipam ao retropropagar por muitos passos. LSTMs atenuam, mas não eliminam o problema.

❌ Dependências de longo alcance são difíceis de aprender

## Contexto comprimido

Toda a história da sequência é comprimida em um único vetor  $h_t$ . Informação de tokens distantes se dilui progressivamente.

❌ Em sequências longas,  $h_t$  quase não carrega informação dos primeiros tokens

**Transformer** (Vaswani et al., 2017) elimina a recorrência: cada token se conecta diretamente a todos os outros via **self-attention** — processamento totalmente paralelo, contexto global, sem vanishing gradient na atenção.

# O problema da polissemia

**Polissemia:** a mesma palavra tem significados diferentes dependendo do contexto.

## "station"

- *"I arrived at the train **station**."* → estação ferroviária
- *"She works at a radio **station**."* → emissora de rádio

## "bank"

- *"I deposited money at the **bank**."* → banco financeiro
- *"We sat on the river **bank**."* → margem do rio

## O problema para word embeddings estáticos

❌ Word2Vec e GloVe atribuem **um único vetor** por palavra — independente do contexto.

"bank" tem sempre o mesmo embedding, misturando os dois significados.

✅ Precisamos de **representações contextuais**: o embedding de "station" deve depender das palavras ao redor.

"train station" → próximo de termos ferroviários

"radio station" → próximo de termos de mídia

# Tarefa ATIS — Slot Filling

**ATIS (Airline Travel Information System):** classificar cada palavra de uma frase em categorias semânticas (*slots*).

|         |   |      |    |     |      |        |    |       |       |     |        |    |        |    |       |       |       |         |
|---------|---|------|----|-----|------|--------|----|-------|-------|-----|--------|----|--------|----|-------|-------|-------|---------|
| Frase:  | I | want | to | fly | from | boston | at | 7     | am    | and | arrive | in | denver | at | 11    | in    | the   | morning |
| Rótulo: | O | O    | O  | O   | O    | B-from | O  | B-dep | I-dep | O   | O      | O  | B-arr  | O  | B-arr | I-arr | I-arr | I-arr   |

**B-fromloc.city\_name** "boston" = cidade de origem

B = início da entidade

**B-depart\_time.time** "7 am" = horário de partida

I = continuação da entidade

**B-toloc.city\_name** "denver" = cidade de destino

O = fora de qualquer entidade

**Desafio:** o rótulo de cada palavra depende do contexto. "at" antes de "boston" é diferente de "at" antes de "7 am". O modelo precisa olhar para os arredores de cada token para atribuir o rótulo correto.

# Três requisitos para uma solução

## 1. Saída do mesmo comprimento que a entrada

Para slot filling, cada token de entrada precisa de um rótulo correspondente.

Entrada: N tokens → Saída: N rótulos

## 2. Capturar contexto circundante

A representação de cada palavra deve levar em conta todas as outras palavras da sequência.

"station" em "train station" ≠ "station" em "radio station"

## 3. Capturar a ordem das palavras

"gato comeu rato" ≠ "rato comeu gato"

A posição de cada token carrega informação semântica essencial.

**RNNs atendem parcialmente** — processam tokens em ordem e mantêm estado oculto. Mas o contexto futuro é ignorado (exceto se bidirecional) e dependências longas sofrem vanishing gradient.

**O Transformer** resolve os três requisitos de forma paralela e eficiente via **self-attention**.

# Parte 2 — Self-Attention



# Intuição: representação contextual

**Ideia central:** a nova representação de uma palavra é uma **média ponderada** dos embeddings de todas as outras palavras.

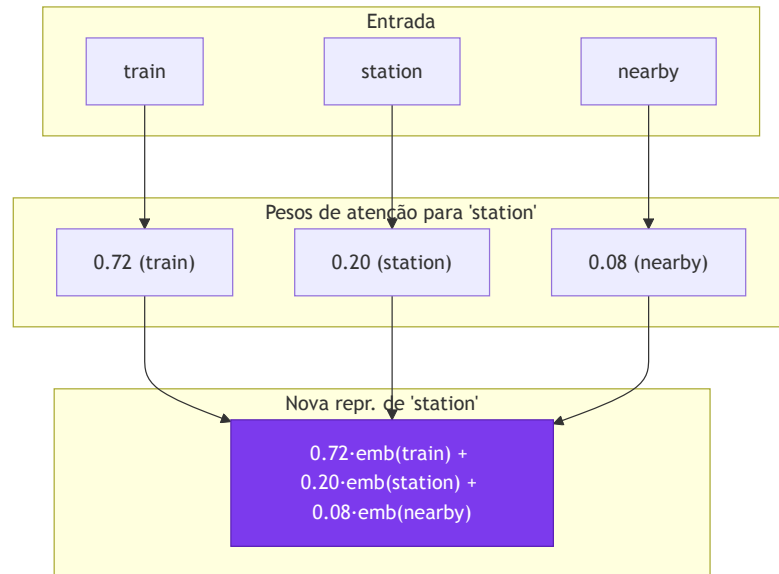
O peso de cada palavra é **proporcional à similaridade** com a palavra alvo.

**Exemplo: "station" em "train station"**

- Alta similaridade com "train" → alto peso
- Baixa similaridade com "the", "at" → peso baixo
- Nova representação de "station" se move para perto de "train" no espaço vetorial

Antes (embedding estático): "station" = ponto fixo no espaço

Depois (self-attention): "station" = combinação pesada de seu



# Como calcular a similaridade

Similaridade entre palavra i e palavra j =  $\mathbf{w}_i \cdot \mathbf{w}_j$

**Similaridade = produto interno (dot product) dos embeddings**

Para calcular o peso que a palavra j recebe ao representar a palavra i, usa-se o produto interno dos seus embeddings.

## Por que produto interno?

Vetores que apontam na mesma direção têm produto interno alto; vetores ortogonais têm produto zero. O produto interno mede o **alinhamento semântico** entre os embeddings — palavras que co-ocorrem em contextos similares terão embeddings alinhados após o treino. A fórmula completa (com softmax e escala) vem nos próximos slides.

# Por que o produto interno captura similaridade?

$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos \theta$  — mede o **alinhamento direcional** entre dois vetores

$\theta \approx 0^\circ \rightarrow$  **produto alto**

"cat" e "kitten" aparecem nos mesmos contextos  $\rightarrow$  embeddings apontam na mesma direção  $\rightarrow$  alta similaridade

$\theta = 90^\circ \rightarrow$  **produto zero**

"cat" e "quantum" raramente co-ocorrem  $\rightarrow$  embeddings ortogonais  $\rightarrow$  sem relação

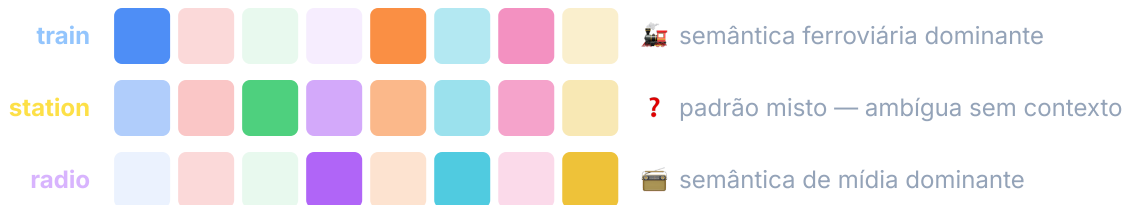
$\theta = 180^\circ \rightarrow$  **produto negativo**

Antônimos podem ter embeddings em direções opostas

**Por que os embeddings ficam alinhados?** Durante o treino, palavras que co-ocorrem em contextos semelhantes recebem gradientes similares  $\rightarrow$  seus vetores convergem para a mesma direção no espaço d-dimensional. O produto interno é **eficiente** ( $O(d)$  e diferenciável) e captura exatamente esse alinhamento.

# Auto-atenção: embedding como "impressão digital"

Frase: "train station nearby the radio station" — mesma palavra, dois sentidos



**Similaridade (dot product):** palavras com padrões parecidos recebem maior peso — o contexto vizinho determina qual sentido de "station" prevalece



# Fórmula do Self-Attention

$$\text{Attention}(\mathbf{W}) = \text{softmax}\left(\frac{\mathbf{W}\mathbf{W}^\top}{\sqrt{d}}\right) \mathbf{W}$$

## Interpretação passo a passo

- $\mathbf{W}$  ( $N \times d$ ): matriz de embeddings das  $N$  palavras
- $\mathbf{W} \cdot \mathbf{W}^\top$  ( $N \times N$ ): todos os produtos internos entre pares de palavras
- Dividir por  $\sqrt{d}$ : estabiliza gradientes (evita softmax saturada)
- softmax linha a linha: pesos de atenção somam 1 para cada palavra
- Multiplicar por  $\mathbf{W}$ : combinar os embeddings com os pesos

## Por que dividir por $\sqrt{d}$ ?

Com embeddings de dimensão  $d$  grande, os produtos internos têm magnitude alta — isso satura o softmax (gradientes  $\approx 0$ ). Escalar por  $1/\sqrt{d}$  mantém a variância dos scores em torno de 1, independente de  $d$ .

**Eficiência computacional:** toda a operação —  $N^2$  produtos internos, escala, softmax e combinação — é expressa como multiplicações de matrizes densas. GPUs executam isso em paralelo para todas as palavras simultaneamente, sem nenhum loop explícito.

**Resultado:** uma nova matriz  $N \times d$  onde cada linha é uma representação contextualizada da palavra correspondente — incorporando informação de

# Exemplo: self-attention passo a passo (d=2)

"train station nearby the radio station" — "station" aparece 2× com embedding inicial idêntico

Embeddings iniciais W (4×2)

|                  | d <sub>1</sub> | d <sub>2</sub> |
|------------------|----------------|----------------|
| tr.              | 0.9            | 0.1            |
| sta <sub>1</sub> | 0.6            | 0.6            |
| ra.              | 0.1            | 0.9            |
| sta <sub>2</sub> | 0.6            | 0.6            |

sta<sub>1</sub> e sta<sub>2</sub> têm embedding **idêntico** 0.6, 0.6

Atenção:  $\text{softmax}(W \cdot W^T \div \sqrt{2})$

|                  | tr.        | sta <sub>1</sub> | ra.        | sta <sub>2</sub> |
|------------------|------------|------------------|------------|------------------|
| tr.              | <b>0.3</b> | 0.25             | 0.19       | 0.25             |
| sta <sub>1</sub> | 0.24       | <b>0.26</b>      | 0.24       | 0.26             |
| ra.              | 0.19       | 0.25             | <b>0.3</b> | 0.25             |
| sta <sub>2</sub> | 0.24       | 0.26             | 0.24       | <b>0.26</b>      |

🔄 linhas sta<sub>1</sub> = sta<sub>2</sub>: pesos de atenção idênticos → mesma representação de saída!

# Parte 3 — Positional Encoding

# O problema: sem posição, sem ordem

## Self-attention é invariante à permutação:

Se embaralharmos a ordem das palavras, os pesos mudam mas a operação não distingue "posição 1" de "posição 5".

✗ Sem modificações, o Transformer trata como equivalentes:

"gato comeu rato" = "rato comeu gato" = "comeu gato rato"

O modelo perde totalmente a informação de ordem.

## Por que isso importa?

**Sujeito vs. objeto** — "João viu Maria" ≠ "Maria viu João": a posição determina quem age.

**Modificadores** — "cachorro grande bravo" ≠ "bravo grande cachorro": a ordem altera a construção gramatical.

**Slot filling** — em "fly from boston at 7am", a posição de "boston" após "from" define seu rótulo como origem.



# Solução: Positional Encoding

**Ideia:** somar um vetor posicional ao embedding de cada palavra antes de entrar no Transformer.

$$\mathbf{x}'_i = \mathbf{x}_i + \mathbf{p}_i$$

## Exemplo numérico:

| Token | Embedding  | + Pos. Enc.  | = Entrada     |
|-------|------------|--------------|---------------|
| "cat" | (0.5, 7.1) | + (1.3, 3.9) | = (1.8, 11.0) |
| "sat" | (1.2, 5.3) | + (6.3, 3.7) | = (7.5, 9.0)  |

As entradas finais diferem mesmo se "cat" e "sat" tivessem embeddings idênticos.

## Duas abordagens:

### Learned (aprendido):

- Cada posição 0...N-1 tem um vetor de embedding treinável
- Flexível, mas limitado ao tamanho máximo de sequência visto no treino
- Usado em BERT, GPT

### Sinusoidal (Vaswani et al., 2017):

- $p(i, 2k) = \sin(i / 10000^{(2k/d)})$
- $p(i, 2k+1) = \cos(i / 10000^{(2k/d)})$
- Generaliza para sequências mais longas que as do treino

# Parte 4 — Multi-Head Attention

# Projeções Q, K, V — tornando a atenção treinável

$$\text{score}(i, j) = \text{softmax}\left(\frac{(A_Q \mathbf{w}_i) \cdot (A_K \mathbf{w}_j)}{\sqrt{d_k}}\right) \quad \text{saída}_i = \sum_j \text{score}(i, j) \cdot A_V \mathbf{w}_j$$

## Por que não usar $W \cdot W^T$ diretamente?

Se Query e Key forem o mesmo vetor  $\mathbf{w}_i$ , o modelo só pode aprender a similaridade entre os embeddings originais — sem flexibilidade.

Com  $A_Q$ ,  $A_K$ ,  $A_V$  separados:

- $A_Q$  transforma  $\mathbf{w}_i$  em uma "consulta" (o que estou buscando?)
- $A_K$  transforma  $\mathbf{w}_j$  em uma "chave" (o que tenho para oferecer?)
- $A_V$  transforma  $\mathbf{w}_j$  em um "valor" (o que devo contribuir?)

## Nomes Q, K, V vêm de sistemas de recuperação:

Query (Q): a pergunta que faço

Key (K): o índice do que está disponível

Value (V): o conteúdo associado

A atenção encontra quais Keys são mais similares à Query e retorna uma combinação pesada dos Values correspondentes.

As matrizes  $A_Q$ ,  $A_K$ ,  $A_V$  são aprendidas via backprop — o modelo decide o que é "relevante" para cada tarefa.

# Multi-Head Attention

## Problema com uma única cabeça:

Uma só cabeça de atenção aprende um único padrão de similaridade. Mas palavras se relacionam de múltiplas formas simultaneamente.

**Cabeça 1** — Concordância de tempo verbal

"correu" → conecta com "ontem", "rapidamente"

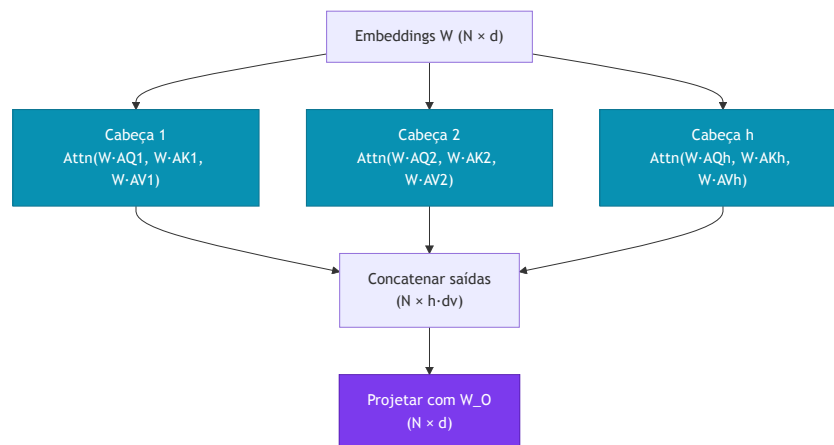
**Cabeça 2** — Relações entre entidades

"presidente" → conecta com o país que governa

**Cabeça 3** — Tom e polaridade

"brilhante" → conecta com outros adjetivos positivos

## Arquitetura Multi-Head



Cada cabeça tem suas próprias matrizes  $A_Q$ ,  $A_K$ ,  $A_V$  aprendíveis, capturando um padrão de atenção independente. A projeção  $W_O$

# Parte 5 — Encoder Completo

# Conexões Residuais

**Problema:** Transformers profundos (muitas camadas) sofrem de gradientes que diminuem ao passar por cada transformação não-linear.

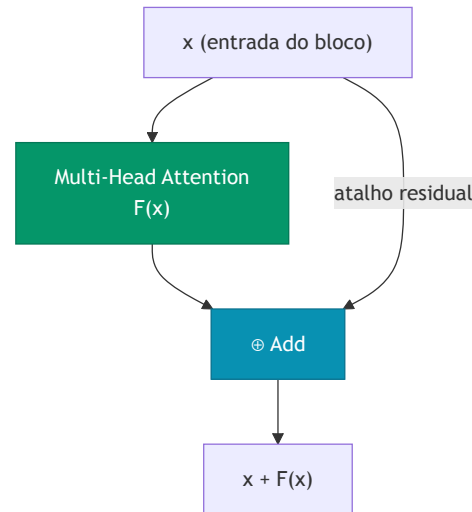
**Solução:** somar a entrada do bloco à sua saída.

Saída do bloco =  $F(x) + x$

onde  $F(x)$  é a transformação aprendida (atenção ou FFN).

**Por que funciona?**

- O gradiente pode fluir diretamente pela conexão aditiva
- A rede aprende apenas a **diferença** (resíduo) em relação à entrada
- Permite treinar redes muito profundas (ResNets usam o mesmo princípio)



Em prática: a conexão residual é aplicada tanto após a atenção quanto após a FFN. Isso permite empilhar dezenas de camadas sem degradação de gradiente.

# Layer Normalization

**Problema:** após as transformações lineares e não-lineares, os vetores de embedding podem ter escala e média muito variáveis — dificultando o treinamento.

**Layer Normalization:** normalizar cada embedding individualmente.

Para um vetor  $h$  de dimensão  $d$ :

$$\text{média} = (1/d) \times \text{soma}(h_i)$$

$$\text{std} = \text{raiz}( (1/d) \times \text{soma}((h_i - \text{média})^2) )$$

$$h_{\text{norm}} = (h - \text{média}) / \text{std}$$

$$h_{\text{out}} = \gamma \times h_{\text{norm}} + \beta$$

## LayerNorm vs BatchNorm

**BatchNorm** Normaliza ao longo da dimensão do batch — problemático para sequências de tamanho variável

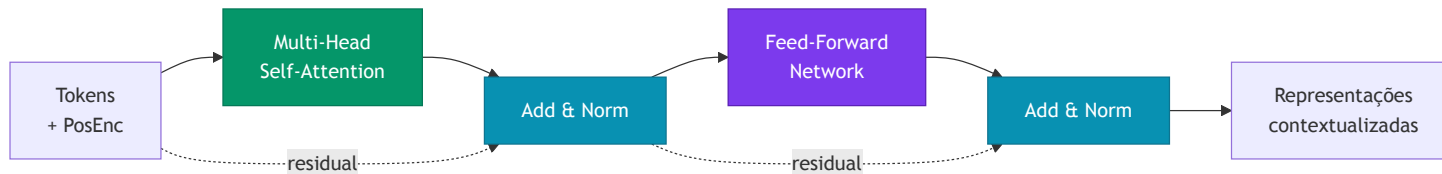
**LayerNorm** Normaliza ao longo das dimensões do embedding — independente do tamanho do batch ou sequência

**gamma e beta** são parâmetros aprendíveis (escala e deslocamento) — permitem ao modelo recuperar a escala original se necessário.

**Resultado:** pesos ficam estáveis durante o treino, acelerando a convergência mesmo com learning rates maiores.

# Bloco Encoder Completo

Cada bloco: **MHA** → **Add & Norm** → **FFN** → **Add & Norm** (N vezes; BERT-base: N = 12)



**N camadas empilhadas:** cada bloco MHA → Add&Norm → FFN → Add&Norm é repetido N vezes (ex: BERT-base: N=12).

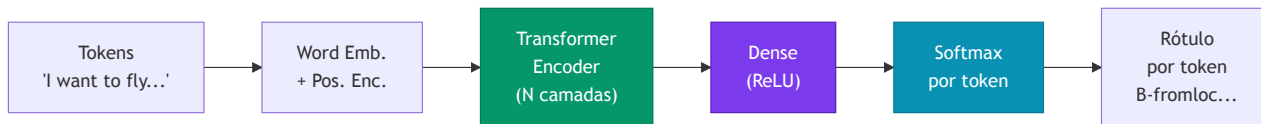
**Pesos otimizados:** embeddings posicionais, embeddings de palavras, A\_Q/A\_K/A\_V de cada cabeça, pesos da FFN, gamma/beta do LayerNorm.



# Parte 6 — Aplicações

# Slot Filling com Transformer

Pipeline completo para classificação token-a-token:



**Entrada** Cada token da frase é convertido para embedding e somado ao encoding posicional correspondente.

**Encoder** O Transformer produz uma representação contextual para cada token — cada vetor "viu" todos os outros tokens.

**Classificação** Uma camada densa + softmax por token produz uma distribuição de probabilidade sobre os rótulos para cada posição.

# Tipos de Tarefas com Transformers

## 1. Classificação de Sequência

Usar o token `[CLS]` ou média dos embeddings finais.

```
CLS] frase... [SEP
```

↓

Dense → Softmax

↓

rótulo da frase

Ex: análise de sentimento, detecção de spam, classificação de tópico.

## 2. Rotulagem de Sequência

Usar o embedding de cada token individualmente.

```
tok1 tok2 tok3
```

↓ ↓ ↓

Dense → Softmax × N

↓ ↓ ↓

```
lab1 lab2 lab3
```

Ex: NER, slot filling, POS tagging.

## 3. Geração de Sequência

Usar Transformer decoder com atenção causal (cada token só pode ver tokens anteriores).

```
[START] → tok1 → tok2 → tok3
```

Ex: tradução automática, sumarização, chatbots.

`[CLS]` (*Classification*) — token especial inserido no início de toda sequência. Após o encoder, seu embedding agrega informação de toda a frase e é usado como representação global para classificação.

`[SEP]` (*Separator*) — token especial que marca o fim de uma sentença ou separa duas sentenças em tarefas de par (ex: inferência textual). Sinaliza ao modelo onde uma sequência termina.

# Geração de Texto: temperatura e top-k

Em modelos **decoder**, a softmax produz uma **distribuição sobre o vocabulário** a cada passo. Como escolher o próximo token?

## Greedy ( $\tau \rightarrow 0$ )

Sempre escolhe o token mais provável. Determinístico e rápido, mas tende à repetição.

## Temperatura ( $\tau$ )

$\text{softmax}(\text{logits} / \tau)$  —  $\tau < 1 \rightarrow$  mais focado;  $\tau > 1 \rightarrow$  mais criativo/aleatório;  $\tau = 1 \rightarrow$  distribuição original do modelo.

## Top-k sampling

Descarta todos exceto os k mais prováveis antes do softmax.  $k = 1 =$  greedy;  $k = 50$  é valor comum na prática.

## Top-p (nucleus sampling)

Mantém o menor conjunto com probabilidade acumulada  $\geq p$  — k varia: menor quando o modelo está confiante.

**Na prática (APIs de LLMs):** temperatura=0.7, top-p=0.9 são valores típicos. Valores altos  $\rightarrow$  mais variado e criativo; valores baixos  $\rightarrow$  mais determinístico e factual.

# Código PyTorch — Transformer Encoder

```
import torch
import torch.nn as nn

# Um bloco do encoder: Multi-Head Attention + FFN + Add&Norm
encoder_layer = nn.TransformerEncoderLayer(
    d_model=512,          # dimensão dos embeddings
    nhead=8,             # número de cabeças de atenção
    dim_feedforward=2048, # dimensão interna da FFN
    dropout=0.1,
    batch_first=True     # formato (batch, seq, d_model)
)

# Empilhar N blocos encoder
encoder = nn.TransformerEncoder(
    encoder_layer=encoder_layer,
    num_layers=6          # N = 6 blocos (como no paper original)
)

# Embeddings de entrada: palavras + posição
embed = nn.Embedding(vocab_size, 512)
pos_enc = nn.Embedding(max_seq_len, 512) # learned positional

# Forward pass
x = embed(tokens) + pos_enc(positions) # (batch, seq, 512)
```

# Recapitulando

## Motivação

- Polisssemia exige representações contextuais
- Slot filling: rótulo de cada token depende do contexto
- 3 requisitos: mesmo comprimento, contexto, ordem

## Self-Attention

- Representação = média ponderada pelos embeddings
- Peso proporcional ao produto interno (similaridade)
- Softmax normaliza; divisão por  $\sqrt{d}$  estabiliza

## Multi-Head

- Múltiplas cabeças capturam padrões diferentes
- Cada cabeça:  $A_Q$ ,  $A_K$ ,  $A_V$  independentes
- Concatenar e projetar com  $W_O$

## Positional Encoding

- Sem encoding: atenção é invariante à ordem
- Somar embedding posicional ao de palavra
- Learned (BERT) ou sinusoidal (original)

## Encoder Completo

- Q/K/V tornam a atenção tunável via backprop
- Residual: gradientes fluem sem atenuação
- LayerNorm: pesos estáveis; N blocos empilhados

## Próxima aula

- BERT: pré-treino e fine-tuning
- GPT: geração autorregressiva
- Aplicações práticas: QA, sumarização

# Próxima aula

## Aula 7 — BERT e Modelos de Linguagem de Grande Escala

- Pré-treino: Masked LM e Next Sentence Prediction
- Fine-tuning para tarefas downstream
- GPT e geração autorregressiva
- Comparativo BERT vs GPT vs T5
- ChatGPT e modelos de instrução

## Para esta semana

- Notebook `lec06-transformers.ipynb` :
  - Implementar self-attention do zero em NumPy
  - Usar `nn.TransformerEncoder` para slot filling
  - Visualizar os pesos de atenção de diferentes cabeças
  - Comparar Transformer vs LSTM bidirecional no ATIS

# Obrigado! Perguntas?

CCM-109 · Deep Learning · UFABC